

Spring Boot in the Cloud

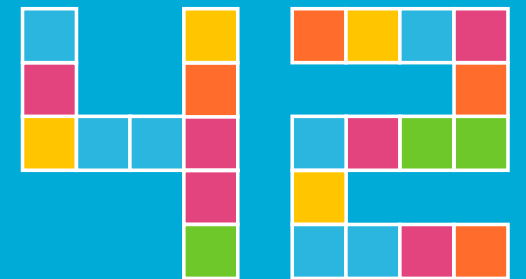
Advanced Optimization Deep Dive

Patrick Baumgartner

42talents GmbH, Zürich, Switzerland

@patbaumgartner

patrick.baumgartner@42talents.com



TALENTS

Abstract

Spring Boot in the Cloud: Advanced Optimization Deep Dive

Spring Boot makes it easy to launch cloud-ready applications, but building truly lean and efficient cloud services requires going beyond the basics. In this three-hour deep dive, you'll immerse yourself in advanced Spring Boot techniques to optimize your applications for the cloud.

We will examine in depth how to reduce memory usage, improve startup times, and streamline your Spring Boot apps for modern cloud environments. Topics include Spring AOT, classpath exclusions, lazy beans, actuator configuration, custom JVM options, and additional state-of-the-art tools.

This session combines short presentations, live coding, and practical hands-on exercises. You will experiment with advanced configurations, observe real-time results, and troubleshoot performance challenges alongside the instructor. Real-world scenarios and best practices will help you immediately apply these techniques to your own projects.

Whether you want to fine-tune your CI/CD pipeline or push Spring Boot to its limits in production, you will leave with actionable skills, a collection of code samples, and a deeper understanding of cloud-native application optimization. Get ready for an in-depth, practical exploration of building lean Spring Boot applications for the cloud.

Spring Boot in the Cloud

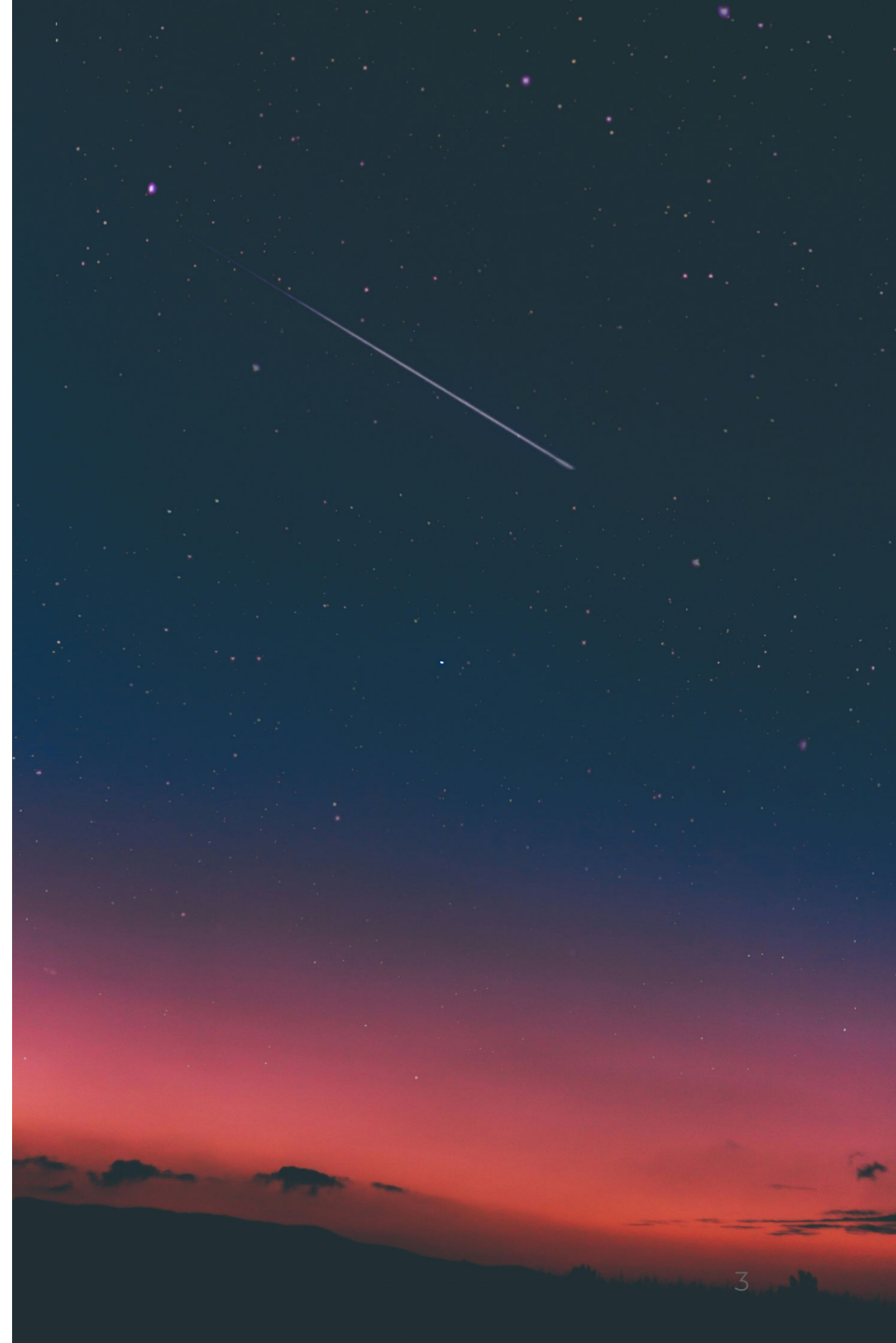
Advanced Optimization Deep Dive

Patrick Baumgartner

42talents GmbH, Zürich, Switzerland

@patbaumgartner

patrick.baumgartner@42talents.com



Spring Boot in the Cloud

Advanced Optimization Deep Dive

Patrick Baumgartner

42talents GmbH, Zürich, Switzerland

@patbaumgartner

patrick.baumgartner@42talents.com

bit.ly/48i5U9x



WARNING:

Numbers shown in this talk are **not** based on **real data** but **only estimates** and **assumptions** made by the **author** for **educational purposes** only.

Introduction



Patrick Baumgartner

Technical Agile Coach @ **42talents**

My focus is on the **development of software solutions *with* humans.**

Coaching, Architecture, Development, Reviews, and Training.

Lecturer @ **Zurich University of Applied Sciences ZHAW**

Co-Organizer of **Voxxed Days Zurich, JUG Switzerland**, Java Champion, Oracle ACE Pro Java ...

[@patbaumgartner](https://twitter.com/patbaumgartner)

What's the challenge?

Why does this matter?

I ❤️ Java 🤗 & Spring Boot 🌿



Introduction to Optimization Goals

Questions ...

Think about your current project.

- What would you optimise your application for?
- What are the key metrics?
- How would you measure them?

3' in pairs

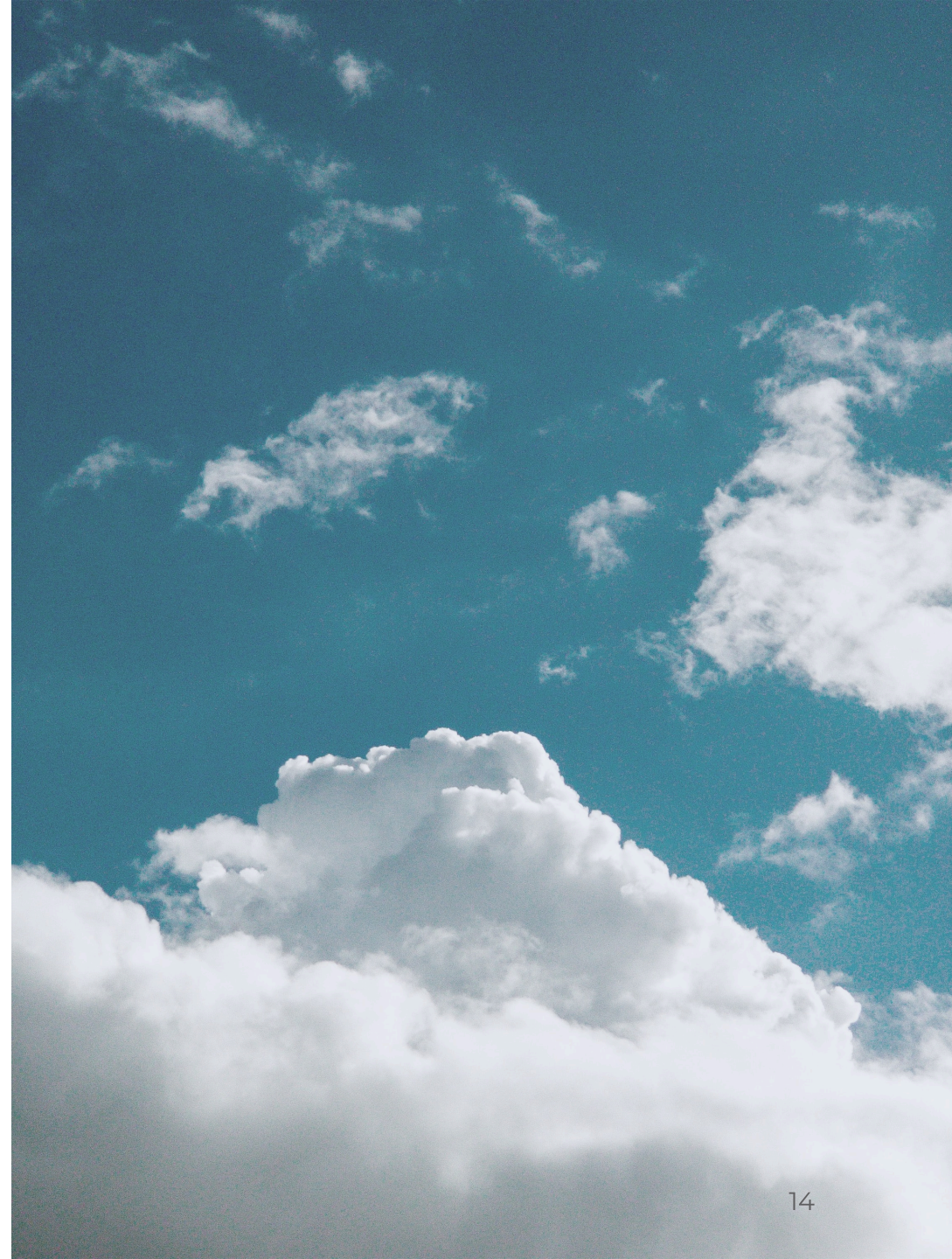
Measures

- Use Case
- Build Time
- Image Size
- Startup Time
- CPU Usage
- Memory Usage
- Throughput / Requests per Second
- Latency / Response Time

Requirements

When Deploying to a Cloud

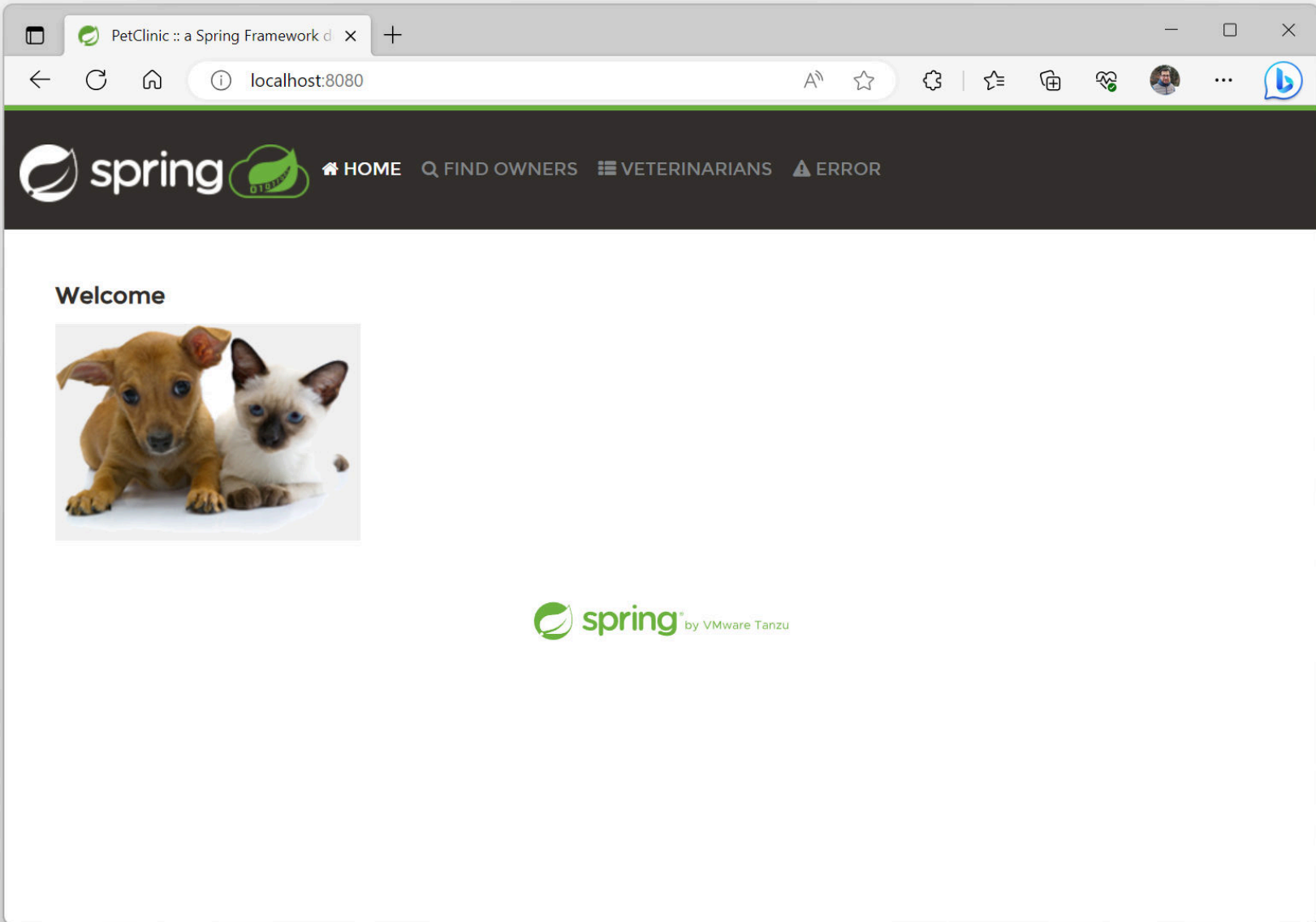
- How many vCPUs needs my application?
- How much RAM does it need?
- How much storage does it need?
- What technology stack should I use?
- What type of application do we build?

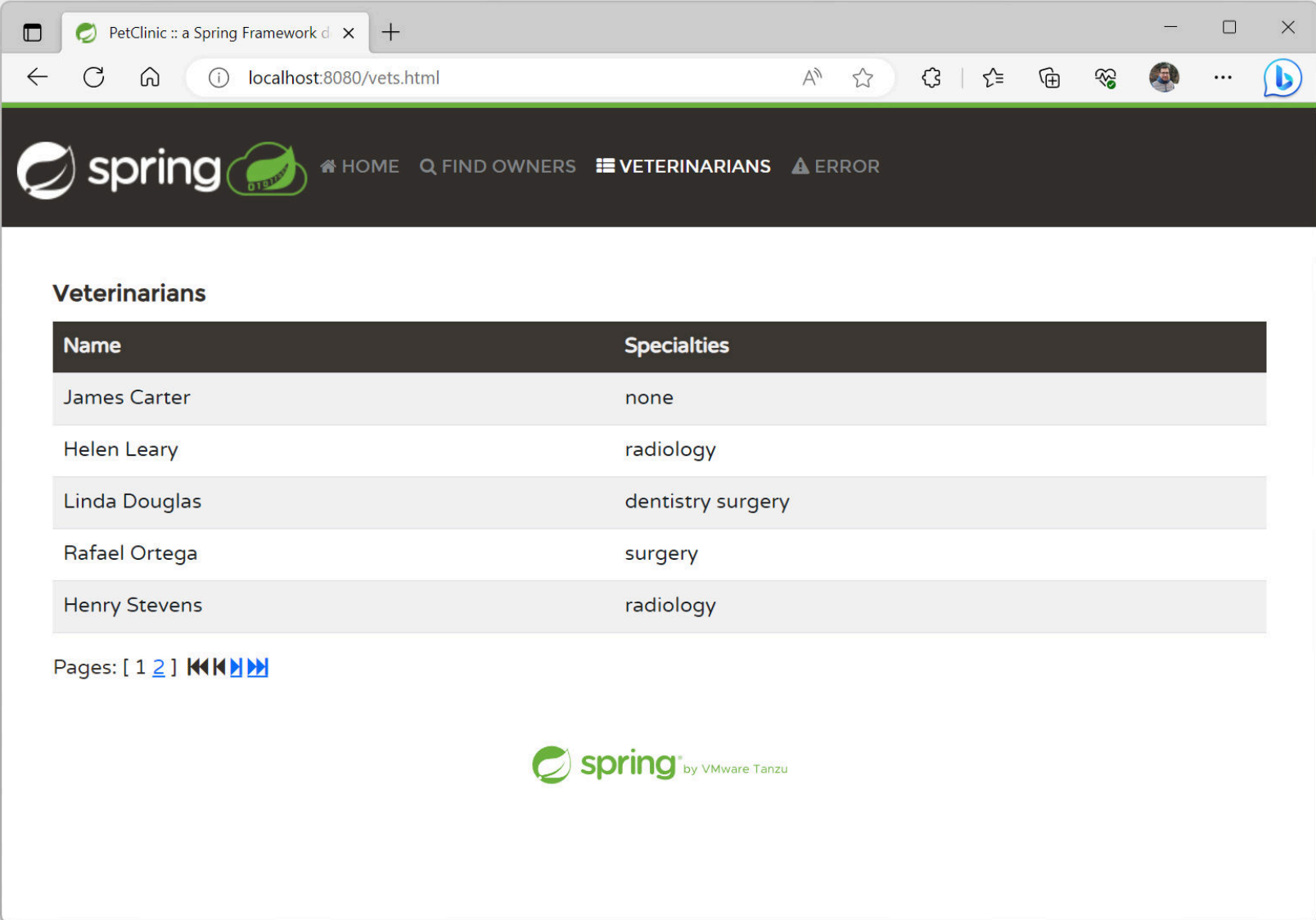


Agenda

Agenda

- Spring PetClinic & Baseline for comparison
- JVM Optimisations
- Spring Boot Optimisations
- Application Optimisations
- Other Runtimes
- Conclusions
- Some simple optimisations applied (OpenJDK examples)





Spring Petclinic Community

- spring-framework-petclinic
- spring-petclinic-angular(js)
- spring-petclinic-rest
- spring-petclinic-graphql
- spring-petclinic-microservices
- spring-petclinic-data-jdbc
- spring-petclinic-cloud
- spring-petclinic-mustache
- spring-petclinic-kotlin
- spring-petclinic-reactive
- spring-petclinic-hilla
- spring-petclinic-angularjs
- spring-petclinic-vaadin-flow
- spring-petclinic-reactjs
- spring-petclinic-htmx
- spring-petclinic-istio
- ...

Projects on GitHub: <https://github.com/spring-petclinic>

NO!

The official Spring PetClinic!  
Based on Spring Boot, Caffeine,
Thymeleaf, Spring Data JPA, H2 and
Spring MVC ...

Project on GitHub: <https://github.com/spring-projects/spring-petclinic>

Limitations

- Limited to the data available
- Limited to the scope of the study
- Limited to the time period of the study
- Limited to the geographical area of the study
- Limited to the methods used
- Limited to the sample size
- Limited to the accuracy of the data
- Limited to the reliability of the data
- Limited to the validity of the data
- Limited to the generalizability of the data
- Limited to the applicability of the data
- Limited to the interpretability of the data
- Limited to the comparability of the data
- Limited to the consistency of the data
- Limited to the completeness of the data
- Limited to the timeliness of the data
- Limited to the relevance of the data
- Limited to the availability of the data
- Limited to the accessibility of the data
- Limited to the transparency of the data
- Limited to the accountability of the data
- Limited to the responsibility of the data
- Limited to the integrity of the data
- Limited to the confidentiality of the data
- Limited to the security of the data
- Limited to the privacy of the data
- Limited to the protection of the data
- Limited to the preservation of the data
- Limited to the archiving of the data
- Limited to the backup of the data
- Limited to the recovery of the data
- Limited to the restoration of the data
- Limited to the migration of the data
- Limited to the integration of the data
- Limited to the interoperability of the data
- Limited to the compatibility of the data
- Limited to the portability of the data
- Limited to the scalability of the data
- Limited to the flexibility of the data
- Limited to the adaptability of the data
- Limited to the robustness of the data
- Limited to the resilience of the data
- Limited to the recoverability of the data
- Limited to the sustainability of the data
- Limited to the manageability of the data
- Limited to the observability of the data
- Limited to the auditability of the data
- Limited to the traceability of the data
- Limited to the accountability of the data
- Limited to the responsibility of the data
- Limited to the integrity of the data
- Limited to the confidentiality of the data
- Limited to the security of the data
- Limited to the privacy of the data
- Limited to the protection of the data
- Limited to the preservation of the data
- Limited to the archiving of the data
- Limited to the backup of the data
- Limited to the recovery of the data
- Limited to the restoration of the data
- Limited to the migration of the data
- Limited to the integration of the data
- Limited to the interoperability of the data
- Limited to the compatibility of the data
- Limited to the portability of the data
- Limited to the scalability of the data
- Limited to the flexibility of the data
- Limited to the adaptability of the data
- Limited to the robustness of the data
- Limited to the resilience of the data
- Limited to the recoverability of the data
- Limited to the sustainability of the data
- Limited to the manageability of the data
- Limited to the observability of the data
- Limited to the auditability of the data
- Limited to the traceability of the data

Limitations

- Docker stats prints wrong values for memory usage, use docker top instead. (See: <https://quarkus.io/guides/performance-measure#measuring-memory-correctly-on-docker>)

```
docker top <CONTAINER ID> -o pid,rss,args
```

- Thermal throttling of your notebook CPU may affect your results

Benchmark Steps

Build with Maven

Set the Java version and build the OCI container image.

```
sdk use java 17.0.17-librca
java -version

build_time=$(./mvnw clean spring-boot:build-image \
  | grep "Total time:" \
  | awk '{print $4 $5}')

echo -e "\033[0;32mBuild time: $build_time\033[0m"

image_size=$(docker images \
  | grep "spring-petclinic" \
  | awk '{print $7}')

echo -e "\033[0;32mImage Size: $image_size\033[0m"
```

Container Startup Time

Run the container with limited memory and measure the startup time.

```
docker container run -d --rm --memory="1g" -p 8080:8080 -t spring-petclinic:4.0.0-SNAPSHOT

# Get the container ID of the running container
container_id=$(docker ps -q)

startup_time=$(docker logs "$container_id"
| grep "Started" \
| awk '{print $14 $15}')

echo -e "\033[0;32mStartup time: $startup_time\033[0m"
```

Memory Usage after Startup

Extract the memory usage of the running container. Convert rss to MB.

```
memory=$(docker top $(docker ps -lq) -o pid,rss,args \  
| tail -n +2 \  
| awk '{ $2=int($2/1024)"MB"; } { print $2; }')  
  
echo -e "\033[0;32mStartup memory usage: $memory\033[0m"
```


Warm-up Application

Warm up the application with for 60s, json output and no UI.

```
oha --no-tui -z60s --disable-keepalive -j "http://localhost:8080/owners?page=2"
```

Load-Test Application

Load test during 60s, json output and no UI.

```
throughput=$(oha --no-tui -z60s --disable-keepalive -j "http://localhost:8080/owners?page=2" 2>&1 \  
| tee $filename \  
| jq '.summary.requestsPerSec')  
  
echo -e "\033[0;32mAvg Throughput: $throughput Requests/sec\033[0m"
```

Memory Usage after Load-Test

Extract the memory usage of the running container. Convert rss to MB.

```
memory=$(docker top $(docker ps -lq) -o pid,rss,args \  
| tail -n +2 \  
| awk '{ $2=int($2/1024)"MB"; }{ print $2; }')  
  
echo -e "\033[0;32mEnd memory usage: $memory\033[0m"
```

Optimisation Experiments

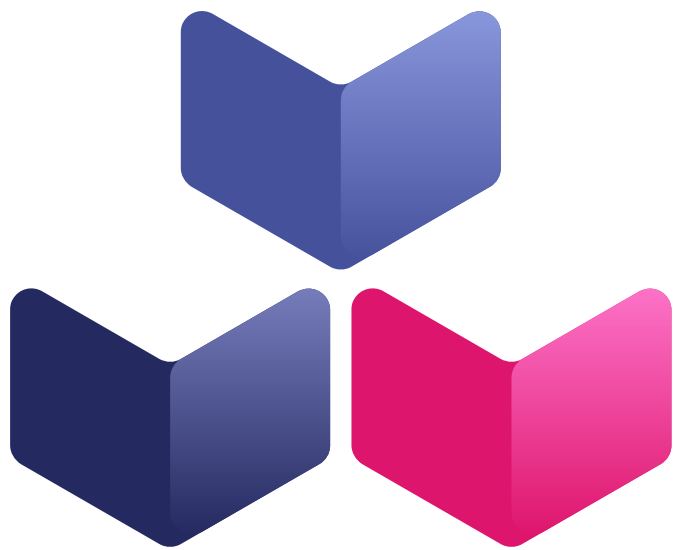
Baseline

Technology Stack

- OCI Container built with Buildpacks
- Java JDK 17 LTS
- Spring Boot 4.0.0
- Testcontainers
- DB migration using SQL scripts

Examination

- Build time
- Startup time
- Resource usage
- Container image size
- Throughput



Buildpacks.io



paketo

buildpacks



**Your app,
in your favorite language,
ready to run in the cloud**



GraalVM



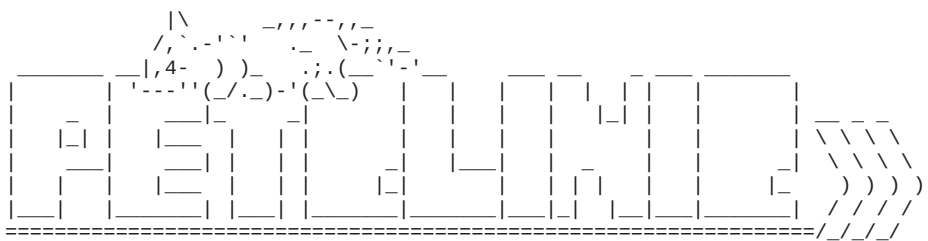
NGINX



```

Calculating JVM memory based on 21866780K available memory
For more information on this calculation, see https://paketo.io/docs/reference/java-reference/#memory-calculator
Calculated JVM Memory Configuration: -XX:MaxDirectMemorySize=10M -Xmx21224904K -XX:MaxMetaspaceSize=129875K -XX:ReservedCodeCacheSize=240M -Xss1M
(Total Memory: 21866780K, Thread Count: 250, Loaded Class Count: 20516, Headroom: 0%)
Enabling Java Native Memory Tracking
Adding 146 container CA certificates to JVM truststore
Spring Cloud Bindings Enabled
Picked up JAVA_TOOL_OPTIONS: -Djava.security.properties=/layers/paketo-buildpacks_bellsoft-liberica/java-security-properties/java-security.properties
-XX:+ExitOnOutOfMemoryError -XX:MaxDirectMemorySize=10M -Xmx21224904K -XX:MaxMetaspaceSize=129875K -XX:ReservedCodeCacheSize=240M -Xss1M
-XX:+UnlockDiagnosticVMOptions -XX:NativeMemoryTracking=summary -XX:+PrintNMTStatistics -Dorg.springframework.cloud.bindings.boot.enable=true

```



```
:: Built with Spring Boot :: 4.0.0
```

```

2025-11-22T19:06:13.029Z INFO 1 --- [main] o.s.s.petclinic.PetClinicApplication : Starting PetClinicApplication v4.0.0-SNAPSHOT using Java 25.0.1
with PID 1 (/workspace/BOOT-INF/classes started by cnb in /workspace)
2025-11-22T19:06:13.034Z INFO 1 --- [main] o.s.s.petclinic.PetClinicApplication : No active profile set, falling back to 1 default profile: "default"
2025-11-22T19:06:14.010Z INFO 1 --- [main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2025-11-22T19:06:14.075Z INFO 1 --- [main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 54 ms. Found 3 JPA repository interfaces.
2025-11-22T19:06:14.743Z INFO 1 --- [main] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2025-11-22T19:06:14.757Z INFO 1 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2025-11-22T19:06:14.757Z INFO 1 --- [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.14]
2025-11-22T19:06:14.802Z INFO 1 --- [main] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 1708 ms
2025-11-22T19:06:17.790Z INFO 1 --- [main] o.s.b.a.e.web.EndpointLinksResolver : Exposing 13 endpoints beneath base path '/actuator'
2025-11-22T19:06:17.875Z INFO 1 --- [main] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2025-11-22T19:06:17.884Z INFO 1 --- [main] o.s.s.petclinic.PetClinicApplication : Started PetClinicApplication in 5.316 seconds (process running for 5.86)

```

1000x better than your regular
Dockerfile  ...

... more **Secure**  and maintained by the
Buildpacks community.

See also: <https://buildpacks.io/> and <https://www.cncf.io/projects/buildpacks/>

Friends don't
let friends write
Dockerfiles!

@starbuxman



Benchmarks

Benchmarks

- Build
 - Maven build time
 - Artifact / Container Image size
- Startup
 - Startup time
 - Memory usage
- Throughput & Latency
 - `oha --no-tui -z60s --disable-keepalive <URL>`
 - 60s warmup, 60s measurement
 - Docker container with 2 vCPU and 1 GB RAM

No Optimizing - Baseline JDK 17

- Spring PetClinic (no adjustments)
- Bellsoft Liberica JDK 17.0.16
- Java Memory Calculator

```
sdk use java 17.0.17-librca  
mvn spring-boot:build-image  
docker run -p 8080:8080 -t spring-petclinic:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms

No Optimizing - Baseline JDK 21

- Spring PetClinic (JDK 21 adjustments)
- Bellsoft Liberica JDK 21.0.9
- Java Memory Calculator

```
sdk use java 21.0.9-librca
```

```
mvn -Djava.version=21 spring-boot:build-image \  
    -Dspring-boot.build-image.imageName=spring-petclinic:4.0.0-SNAPSHOT-jdk21
```

```
docker run -p 8080:8080 -t spring-petclinic:4.0.0-SNAPSHOT-jdk21
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
Java 21	75s	305MB	4.473s	273MB	368/s	375MB	113ms	189ms	202ms	299ms	394ms

No Optimizing - Baseline JDK 25

- Spring PetClinic (JDK 25 adjustments)
- Bellsoft Liberica JDK 25.0.1
- Java Memory Calculator

```
sdk use java 25.0.1-librca
```

```
mvn -Djava.version=25 spring-boot:build-image \  
    -Dspring-boot.build-image.imageName=spring-petclinic:4.0.0-SNAPSHOT-jdk25
```

```
docker run -p 8080:8080 -t spring-petclinic:4.0.0-SNAPSHOT-jdk25
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
Java 25	76s	373MB	4.423s	265MB	268/s	391MB	196ms	212ms	291ms	387ms	414ms

JVM Optimisations

-noverify

The verifier is turned off because some of the bytecode rewriting stretches the meaning of some bytecodes - in a way that doesn't bother the JVM, but does bother the verifier.

Warning: The `-Xverify:none` and `-noverify` options are deprecated in JDK 13 and are likely to be removed in a future release.

```
docker run -p 8080:8080 -e "JAVA_TOOL_OPTIONS=-noverify" \  
-t spring-petclinic:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
Java 17	-	-	3.906s	245MB	344/s	376MB	114ms	195ms	215ms	385ms	492ms
Java 21	-	-	3.819s	256MB	381/s	374MB	110ms	188ms	202ms	301ms	397ms
Java 25	-	-	3.872s	253MB	264/s	484MB	197ms	216ms	294ms	392ms	486ms

-XX:+UseCompactObjectHeaders

The Compact Object Headers (JEP 384) feature reduces the memory footprint of Java objects by using a more compact representation for object headers. This can lead to improved cache performance and reduced memory usage.

```
docker run -p 8080:8080 -e "JAVA_TOOL_OPTIONS=-XX:+UseCompactObjectHeaders" \
-t spring-petclinic:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
 Java 25	76s	373MB	4.423s	265MB	268/s	391MB	196ms	212ms	291ms	387ms	414ms
Java 25	-	-	4.41s	263MB	243/s	460MB	201ms	283ms	305ms	406ms	500ms

-XX:TieredStopAtLevel=1

Tiered compilation is enabled by default (since Java 8). Unless explicitly specified, the JVM decides which JIT compiler to use based on our CPU. For multi-core processors or 64-bit VMs, the JVM will select C2.

To disable C2 and use only the C1 compiler with no profiling overhead, we can use the `-XX:TieredStopAtLevel=1` parameter.

```
docker run -p 8080:8080 -e "JAVA_TOOL_OPTIONS=-XX:TieredStopAtLevel=1" \
-t spring-petclinic:4.0.0-SNAPSHOT
```

It will slow down the JIT later at the expense of the saved startup time!

	Build	Image	Startup	Initial RAM	T...	RAM	50%	75%	90%	99%	99.9%
 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
Java 17	-	-	3.654s	221MB	159/s	294MB	308ms	391ms	408ms	501ms	588ms
Java 21	-	-	3.782s	230MB	149/s	305MB	319ms	398ms	441ms	510ms	597ms
Java 25	-	-	3.495s	227MB	185/s	432MB	289ms	318ms	402ms	512ms	605ms

-XX:+UseParallelGC

The Parallel Garbage Collector (Parallel GC) is a high-throughput garbage collector that is designed to take advantage of multiple processors. It performs minor garbage collections in parallel, which can lead to shorter pause times and improved application performance.

```
docker run -p 8080:8080 -e "JAVA_TOOL_OPTIONS=-XX:+UseParallelGC" \
-t spring-petclinic:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	T...	RAM	50%	75%	90%	99%	99.9%
 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
Java 17	-	-	4.47s	312MB	350/s	473MB	114ms	195ms	209ms	313ms	437ms
Java 21	-	-	4.424s	330MB	371/s	488MB	112ms	188ms	201ms	294ms	361ms
Java 25	-	-	4.334s	347MB	265/s	509MB	197ms	212ms	291ms	384ms	406ms

-XX:+UseZGC -XX:+ZGenerational

The Z Garbage Collector (ZGC) is a scalable, low-latency garbage collector. ZGC performs all expensive work concurrently without stopping application threads for more than 10ms, making it suitable for applications that require low latency and/or use a very large heap (multi-terabyte).

```
docker run -p 8080:8080 -e "JAVA_TOOL_OPTIONS=-XX:+UseZGC -XX:+ZGenerational" \
-t spring-petclinic:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	T...	RAM	50%	75%	90%	99%	99.9%
 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
Java 21	-	-	4.6s	511MB	351/s	692MB	115ms	192ms	204ms	301ms	395ms
Java 25	-	-	4.613s	527MB	261/s	677MB	198ms	213ms	293ms	391ms	483ms

See also: <https://wiki.openjdk.org/display/zgc/Main>

VM Options Explorer

<https://chriswhocodes.com>

VM Options Explorer - OpenJDK11

<https://chriswhocodes.com>

Byte-Me FullJEP JEPMap JEPSearch hsdis JITWatch JaCoLine VM Options Explorer VM Intrinsic Explorer GC Explorer Optimizing Java Thank You!

VM Options Explorer - OpenJDK11 HotSpot

[OpenJDK HotSpot](#)
[Options added/removed between JDKs](#)
OpenJDK options also hosted on [foojay.io](#)

[Alibaba Dragonwell](#)

[Amazon Corretto](#)

[Azul Systems](#)

Platform PrimeZulu

[BellSoft Liberica](#)

[Eclipse Temurin](#)

[JDK-based GraalVM](#)

17212224

JDKNativeJDKNativeJDKNativeJDKNative

[JetBrains Runtime](#)

[Microsoft](#)

[OpenJ9](#)

[Oracle](#)

[SAP SapMachine](#)

6789101112131415161718192021222324

1117192021

Search OpenJDK11 HotSpot Options:

Name	Since	Deprecated	Type	OS	CPU	Component	Default	Availability	Description	Defined in
Show All	Deprecated	Show All	Show All	Show All	Show All	Show All	Show All	Show All		

Spring Boot Optimisations

Lazy Spring Beans (1)

Configure lazy initialisation throughout your application. A Spring Boot property makes all beans lazy by default, initialising them only when needed. You can use `@Lazy` to override this behaviour, e.g. `@Lazy(false)`.

```
docker run -p 8080:8080 \
  -e spring.main.lazy-initialization=true \
  -e spring.data.jpa.repositories.bootstrap-mode=lazy \
  -t spring-petclinic:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
Java 17	-	-	3.809s	236MB	338/s	368MB	117ms	197ms	220ms	387ms	492ms

Lazy Spring Beans (2)

Pros

- Faster startup useful in cloud environments
- Application startup is a CPU-intensive task. Spreads load over time

Cons

- Initial requests may take longer
- Class loading problems and misconfigurations not detected at startup
- Beans creation errors only be found when the bean is loaded

Fixing Spring Boot Config Location

Determine the location of the Spring Boot configuration file(s).
Considered in the following order (`application.properties` and YAML variants)

- Application properties packaged in your jar
- Profile-specific application properties packaged within your jar
- Application properties outside your packaged jar
- Profile-specific application properties outside your packaged jar

```
docker run -p 8080:8080 -e spring.config.location=classpath:application.properties \
  t spring-note:1.0.0 SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
Java 17	-	-	4.404s	254MB	342/s	368MB	116ms	196ms	214ms	382ms	488ms

No Spring Boot Actuators

Don't use actuators if you can afford not to 😊.

- Number of Spring Beans
 - Spring Pet Clinic with actuators: 454
 - Spring Pet Clinic without actuators: 282 🔥

```
sdk use java 17.0.17-librca
mvn spring-boot:build-image
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
🕒 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
Java 17	74s	275MB	3.984s	243MB	376/s	353MB	110ms	189ms	204ms	307ms	407ms
Java 21	74s	302MB	3.926s	252MB	397/s	353MB	108ms	183ms	197ms	289ms	361ms
Java 25	75s	371MB	3.89s	251MB	295/s	561MB	188ms	204ms	281ms	357ms	407ms

Ahead-of-Time Processing (AOT) (1)

Spring AOT is a process that analyses your application at build time and generates an optimised version of it.

As the `BeanFactory` is fully prepared at build time, conditions are also evaluated. E.g. in `Configurations`, `Component- & Entity-Scan`, `@Profile`, `@Conditional`, etc.

Spring AOT serves as a replacement for `spring-context-indexer` since Spring Framework 6.1 and Spring Boot 3.2.

Ahead-of-Time Processing (AOT) (2)

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <env>
        <BP_SPRING_AOT_ENABLED>true</BP_SPRING_AOT_ENABLED>
      </env>
    </image>
  </configuration>
  <executions>
    <execution>
      <id>process-aot</id>
      <goals>
        <goal>process-aot</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

Ahead-of-Time Processing (AOT) (3)

We create a new container image with the AOT-processed application. The Buildpack enables the property `spring.aot.enabled=true`

```
sdk use java 17.0.17-librca  
  
mvn spring-boot:build-image  
  
docker run -p 8080:8080 -e spring.aot.enabled=true \  
-t spring-petclinic-aot:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
Java 17	82s	279MB	3.722s	238MB	343/s	359MB	115ms	195ms	214ms	383ms	488ms
Java 21	81s	306MB	3.564s	245MB	363/s	359MB	114ms	189ms	202ms	299ms	391ms
Java 25	81s	375MB	3.562s	236MB	269/s	658MB	195ms	213ms	293ms	392ms	482ms

Disabling JMX

JMX is `spring.jmx.enabled=false` by default in Spring Boot since 2.2.0 and later. Setting `BPL_JMX_ENABLED=true` and `BPL_JMX_PORT=9999` on the container will add the following arguments to the `java` command.

```
-Djava.rmi.server.hostname=127.0.0.1  
-Dcom.sun.management.jmxremote.authenticate=false  
-Dcom.sun.management.jmxremote.ssl=false  
-Dcom.sun.management.jmxremote.port=9999  
-Dcom.sun.management.jmxremote.rmi.port=9999
```

```
docker run -p 8080:8080 -p 9999:9999 \  
  -e BPL_JMX_ENABLED=false \  
  -e BPL_JMX_PORT=9999 \  
  -e spring.jmx.enabled=false \  
  -t spring-petclinic:4.0.0-SNAPSHOT
```

I ❤️
**Spring Boot 🌿 &
Buildpacks 🚀**

Application Optimisations

Dependency Cleanup (2)

DepClean detects all unused dependencies declared in the `pom.xml` file of a project and creates a `pom-debloated.xml`. The generated report shows possible unused dependencies.

```
<plugin>
  <groupId>se.kth.castor</groupId>
  <artifactId>depclean-maven-plugin</artifactId>
  <version>2.2.0-SNAPSHOT</version>
  <executions>
    <execution>
      <goals>
        <goal>depclean</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

```
mvn se.kth.castor:depclean-maven-plugin:2.2.0-SNAPSHOT:depclean -DfailIfUnusedDirect=true -DignoreScopes=provided,test,runtime,system,import
```

```

-----
D E P C L E A N   A N A L Y S I S   R E S U L T S
-----
USED DIRECT DEPENDENCIES [10]:
  com.h2database:h2:2.4.240:runtime (2 MB)
  com.mysql:mysql-connector-j:9.5.0:runtime (2 MB)
  org.postgresql:postgresql:42.7.8:runtime (1 MB)
  com.github.ben-manes.caffeine:caffeine:3.2.3:compile (996 KB)
  jakarta.xml.bind:jakarta.xml.bind-api:4.0.4:compile (128 KB)
  ...
USED TRANSITIVE DEPENDENCIES [107]:
  org.testcontainers:testcontainers:2.0.2:test (16 MB)
  org.hibernate.orm:hibernate-core:7.1.8.Final:compile (13 MB)
  net.bytebuddy:byte-buddy:1.17.8:runtime (8 MB)
  org.apache.tomcat.embed:tomcat-embed-core:11.0.14:compile (3 MB)
  com.github.docker-java:docker-java-transport-zero-dep:3.7.0:test (2 MB)
  org.aspectj:aspectjweaver:1.9.25:compile (2 MB)
  org.springframework:spring-web:7.0.1:compile (2 MB)
  org.springframework.data:spring-data-jpa:4.0.0:compile (2 MB)
  org.springframework:spring-core:7.0.1:compile (1 MB)
  ...
USED INHERITED DIRECT DEPENDENCIES [0]:
USED INHERITED TRANSITIVE DEPENDENCIES [0]:
POTENTIALLY UNUSED DIRECT DEPENDENCIES [13]:
  org.webjars.npm:bootstrap:5.3.8:compile (1 MB)
  org.webjars.npm:font-awesome:4.7.0:compile (665 KB)
  org.springframework.boot:spring-boot-devtools:4.0.0:compile (208 KB)
  org.springframework.boot:spring-boot-docker-compose:4.0.0:test (123 KB)
  org.springframework.boot:spring-boot-starter-actuator:4.0.0:compile (4 KB)
  org.springframework.boot:spring-boot-starter-rest-client-test:4.0.0:test (4 KB)
  ...
POTENTIALLY UNUSED TRANSITIVE DEPENDENCIES [35]:
  commons-codec:commons-codec:1.19.0:test (365 KB)
  org.springframework.boot:spring-boot-actuator:4.0.0:compile (348 KB)
  org.yaml:snakeyaml:2.5:compile (332 KB)
  org.attoparser:attoparser:2.0.7.RELEASE:compile (240 KB)
  org.springframework.boot:spring-boot-micrometer-metrics:4.0.0:compile (236 KB)
  org.thymeleaf:thymeleaf-spring6:3.1.3.RELEASE:compile (184 KB)
  ...
POTENTIALLY UNUSED INHERITED DIRECT DEPENDENCIES [0]:
POTENTIALLY UNUSED INHERITED TRANSITIVE DEPENDENCIES [0]:
[INFO] Analysis done in 0min 24s

```

Dependency Cleanup (2)

But there are some challenges:

- Component & Entity Scanning through Classpath Scanning
- Spring Boot uses `META-INF/spring-boot/org.springframework.boot.autoconfigure.AutoConfiguration.imports`
- Spring XML Configuration and `web.xml`

```
sdk use java 17.0.17-librca
```

```
mvn spring-boot:build-image
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
Java 17	74s	271MB	3.794s	243MB	383/s	348MB	108ms	189ms	205ms	309ms	438ms
Java 21	74s	299MB	3.738s	242MB	411/s	354MB	107ms	180ms	196ms	290ms	362ms
Java 25	74s	368MB	3.833s	244MB	314/s	554MB	181ms	200ms	256ms	312ms	404ms

More Application Optimisations

- Application Optimisations
- Better Connection Pooling
- Improved Data Caching
- Database Queries Optimisations
- Batch Processing Optimisations
- Reduce Garbage Collection
- Monitor and Tune JVM Settings
- Reduce Overall Memory Footprint
- Async Logging

Even More JVM Optimisations

JLink (1)

`jlink` assembles and optimises a set of modules and their dependencies into a custom runtime image for your application.

```
$ jlink \  
  --add-modules java.base, ... \  
  --strip-debug \  
  --no-man-pages \  
  --no-header-files \  
  --compress=2 \  
  --output /javaruntime
```

```
$ /javaruntime/bin/java HelloWorld  
Hello, World!
```


JLink (2)

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <env>
        <BP_JVM_JLINK_ENABLED>true</BP_JVM_JLINK_ENABLED>
      </env>
    </image>
  </configuration>
</plugin>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
Java 17	86s	188MB	4.484s	274MB	341/s	380MB	116ms	196ms	215ms	386ms	490ms
Java 21	85s	196MB	4.485s	285MB	367/s	385MB	113ms	189ms	202ms	299ms	394ms
Java 25	84s	199MB	4.385s	280MB	273/s	377MB	195ms	210ms	291ms	388ms	463ms

Java Dependency Analysis Tool

`jdeps` is a Java command-line tool that analyzes class and jar files to list the package-level or class-level dependencies, helping developers understand module dependencies and improve code modularity.

```
jar xvf target/spring-petclinic-4.0.0-SNAPSHOT.jar

jdeps --ignore-missing-deps -q \
      --recursive \
      --multi-release 17 \
      --print-module-deps \
      --class-path 'BOOT-INF/lib/*' \
      target/spring-petclinic-4.0.0-SNAPSHOT.jar
```

```
java.base,java.compiler,java.desktop,java.instrument,java.net.http,java.prefs,java.rmi,java.scripting,java.security.jgss,java.sql.rowset,jdk.jfr,jdk.management,jdk.unsupported
```

JLink (3)

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <env>
        <BP_JVM_JLINK_ENABLED>true</BP_JVM_JLINK_ENABLED>
        <BP_JVM_JLINK_ARGS>--add-modules java.base,java.compiler,java.desktop,java.instrument,
        java.net.http,java.prefs,java.rmi,java.scripting,java.security.jgss,java.sql.rowset,
        jdk.jfr,jdk.management,jdk.unsupported --compress=2 --no-header-files --no-man-pages
        --strip-debug</BP_JVM_JLINK_ARGS>
      </env>
    </image>
  </configuration>
</plugin>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
Java 17	84s	177MB	4.523s	266MB	340/s	379MB	117ms	196ms	235ms	387ms	492ms
Java 21	83s	184MB	4.428s	279MB	360/s	379MB	114ms	191ms	204ms	302ms	399ms
Java 25	82s	187MB	4.453s	275MB	273/s	378MB	195ms	210ms	291ms	389ms	485ms

App CDS (1)

Class Data Sharing (CDS) is a JVM feature that can help reduce the startup time and memory footprint of Java applications. It allows classes to be pre-processed into a shared archive file that can be memory-mapped at runtime.

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <env>
        <BP_JVM_CDS_ENABLED>true</BP_JVM_CDS_ENABLED>
      </env>
    </image>
  </configuration>
</plugin>
```

App CDS (2)

To create the archive, two additional JVM flags must be specified:

- `-XX:ArchiveClassesAtExit=application.jsa` : creates the CDS archive on exit
- `-Dspring.context.exit=onRefresh` : starts and then immediately exits

Once the archive is available, the JVM can be started with the additional flag:

- `-XX:SharedArchiveFile=application.jsa` : to use it

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
Java 17	91s	445MB	2.605s	236MB	414/s	363MB	105ms	183ms	200ms	302ms	403ms
Java 21	90s	472MB	2.643s	245MB	446/s	359MB	103ms	137ms	193ms	290ms	383ms
Java 25	89s	539MB	2.514s	240MB	298/s	385MB	188ms	203ms	278ms	312ms	398ms

I ❤️
**Spring Boot 🌿 &
Buildpacks 🚀**

AOT Caching

Project Leyden is an OpenJDK project that aims to improve the startup time and reduce the memory footprint of Java applications by introducing AOT (Ahead-of-Time) compilation and class data sharing (CDS) enhancements.

Training the AOT cache requires two additional JVM flags:

`-XX:AOTCacheOutput=app.aot -Dspring.context.exit=onRefresh`

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
Java 25	89s	490MB	2.981s	236MB	284/s	372MB	192ms	207ms	287ms	373ms	402ms

Other Runtimes



Unleash the power of Java

Optimized to run Java™ applications cost-effectively in the cloud, Eclipse OpenJ9™ is a fast and efficient JVM that delivers power and performance when you need it most.

Optimized for the Cloud, for microservices and monoliths too!

Faster Startup

Faster Ramp-up, when deployed to cloud

Smaller

Our Story

Eclipse OpenJ9

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <buildpacks>
        <buildpack>paketobuildpacks/eclipse-openj9:latest</buildpack>
        <!-- Used to inherit all the other Java buildpacks -->
        <buildpack>urn:cnb:builder:paketo-buildpacks/java</buildpack>
      </buildpacks>
    </image>
  </configuration>
</plugin>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
Java 17	85s	299MB	4.996s	186MB	115/s	272MB	441ms	494ms	508ms	598ms	685ms
Java 21	87s	318MB	5.076s	193MB	101/s	271MB	505ms	564ms	598ms	690ms	707ms
Java 25	86s	316MB	5.415s	194MB	98/s	269MB	511ms	586ms	602ms	693ms	733ms

GraalVM CE

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <buildpacks>
        <buildpack>paketobuildpacks/graalvm:latest</buildpack>
        <!-- Used to inherit all the other Java buildpacks -->
        <buildpack>urn:cnb:builder:paketo-buildpacks/java</buildpack>
      </buildpacks>
    </image>
  </configuration>
</plugin>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
Java 17	98s	675MB	4.492s	291MB	299/s	397MB	186ms	204ms	285ms	389ms	442ms
Java 21	100s	660MB	4.552s	309MB	308/s	407MB	166ms	202ms	265ms	369ms	470ms
Java 25	96s	848MB	4.502s	322MB	275/s	418MB	193ms	214ms	300ms	409ms	558ms

Oracle JDK

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <buildpacks>
        <buildpack>paketobuildpacks/oracle:latest</buildpack>
        <!-- Used to inherit all the other Java buildpacks -->
        <buildpack>urn:cnb:builder:paketo-buildpacks/java</buildpack>
      </buildpacks>
    </image>
  </configuration>
</plugin>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
Java 21	82s	445MB	4.349s	271MB	360/s	388MB	114ms	190ms	202ms	299ms	394ms
Java 25	83s	490MB	4.407s	266MB	271/s	389MB	195ms	211ms	291ms	386ms	457ms

Azul Zulu

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <buildpacks>
        <buildpack>paketobuildpacks/azul-zulu:latest</buildpack>
        <!-- Used to inherit all the other Java buildpacks -->
        <buildpack>urn:cnb:builder:paketo-buildpacks/java</buildpack>
      </buildpacks>
    </image>
  </configuration>
</plugin>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
Java 17	77s	241MB	4.454s	261MB	346/s	372MB	115ms	195ms	214ms	382ms	491ms
Java 21	75s	265MB	4.17s	273MB	366/s	376MB	113ms	189ms	202ms	299ms	392ms
Java 25	76s	373MB	4.425s	264MB	270/s	442MB	195ms	212ms	293ms	390ms	462ms

Other Buildpack Builders

Bellsoft Buildpack Builder

Bellsoft provides an optimised builder for Spring Boot applications. It uses the Bellsoft Alpaquita, Liberica JDK and the musl C library. A glibc version is also available.

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <builder>bellsoft/buildpacks.builder:musl</builder>
    </image>
  </configuration>
</plugin>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
Java 17 (m)	74s	182MB	4.397s	272MB	356/s	391MB	114ms	192ms	206ms	306ms	406ms
Java 21 (m)	73s	191MB	4.24s	297MB	383/s	397MB	111ms	185ms	198ms	291ms	378ms
Java 25 (m)	70s	197MB	4.262s	292MB	270/s	397MB	195ms	213ms	294ms	391ms	465ms
Java 25 (g)	71s	202MB	4.267s	279MB	279/s	379MB	193ms	207ms	286ms	375ms	403ms

Buildpack Jammy Builder Tiny

It's based on Ubuntu Jammy, and works with current JAVA versions. While the base image (default) is bigger the tiny is intended to be used with GraalVM native images.

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <builder>paketobuildpacks/builder-jammy-java-tiny</builder>
    </image>
  </configuration>
</plugin>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
Java 17	77s	277MB	4.462s	257MB	339/s	368MB	117ms	196ms	233ms	385ms	489ms
Java 21	75s	305MB	4.48s	270MB	361/s	371MB	114ms	191ms	204ms	301ms	395ms
Java 25	76s	373MB	4.482s	270MB	272/s	396MB	195ms	211ms	292ms	390ms	481ms

GraalVM Native Image (CE & Oracle)

A native image is a technology for building Java code into a standalone executable. This executable contains the application classes, classes from its dependencies, runtime library classes and statically linked native code from the JDK. The JVM is packaged in the native image, so there's no need for a Java Runtime Environment on the target system, but the build artifact is platform dependent.

```
mvn -Pnative spring-boot:build-image
```

```
docker run -p 8080:8080 -t spring-petclinic-native:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
CE 25	228s	230MB	0.263s	204MB	612/s	311MB	87ms	99ms	110ms	155ms	173ms
OG 25	314s	226MB	0.187s	176MB	751/s	325MB	73ms	90ms	103ms	147ms	170ms

CRaC - OpenJDK (1)

CRaC (Coordinated Restore at Checkpoint) is a feature that allows you to take a snapshot of the state of a Java application and restart it from that state.

Currently only available from:

- Azul Zulu
- Bellsoft Liberica

The application starts within milliseconds!

.. with many other challenges like: embedded configs and secrets, same OS (Linux) and CPU Architecture, same JVM versions, and more ...

CRaC - OpenJDK (2)

```
export JAVA_HOME=/opt/openjdk-17-crac+6_linux-x64/  
export PATH=$JAVA_HOME/bin:$PATH
```

```
mvn clean verify
```

```
java -XX:CRaCCheckpointTo=crac-files -jar target/spring-petclinic-crac-4.0.0-SNAPSHOT.jar
```

```
jcmd target/spring-petclinic-crac-4.0.0-SNAPSHOT.jar JDK.checkpoint
```

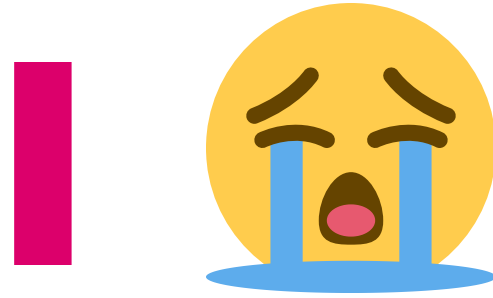
```
java -XX:CRaCRestoreFrom=crac-files
```

CRaC - OpenJDK (3)

CRaC is currently in an experimental state and has the following limitations.

- Since Spring Boot 3.2 CraC support finalised
 - Spring Framework 6.1.0
- Only available for Linux x86 / ARM 64 Bit
- Azul Zulu Build of OpenJDK with CRaC support for development purposes
 - Available for Windows and macOS
 - Simulated checkpoint/restore mechanism for development and testing

Other JVM vendors have similar features, e.g. OpenJ9 with CRIU support.



**No CRaC Buildpacks
for Java & Spring Boot** 🍃

Virtual Threads

A thread is the smallest unit of processing that can be scheduled. It runs concurrently with - and largely independently of - other such units. It is an instance of `java.lang.Thread`. There are two types of threads, platform threads and virtual threads.

```
sdk use java 21.0.9-librca
mvn spring-boot:build-image
docker run -e spring.threads.virtual.enabled=true \
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
Java 21	-	-	4.447s	274MB	594/s	359MB	93ms	96ms	100ms	171ms	201ms
Java 24	-	-	4.624s	262MB	1525/s	380MB	25ms	31ms	33ms	225ms	2052ms
Java 25	-	-	4.421s	265MB	1482/s	401MB	23ms	31ms	33ms	177ms	2328ms

Virtual Threads Java 21

CPU	VT	Startup	Initial RAM	Request/s	RAM	50%	75%	90%	99%	99.9%	99.99%
4 vCPU		6.543s	278MB	293/s	392MB	133ms	207ms	309ms	637ms	941ms	1275ms
4 vCPU		6.479s	288MB	390/s	390MB	114ms	150ms	181ms	194ms	621ms	2346ms
2 vCPU		8.574s	262MB	66/s	352MB	636ms	1007ms	1469ms	2452ms	3312ms	4098ms
2 vCPU		8.359s	262MB	164/s	347MB	296ms	332ms	400ms	572ms	868ms	2204ms
1 vCPU		23.083s	263MB	22/s	348MB	2133ms	3034ms	3967ms	6200ms	8011ms	8361ms
1 vCPU		23.159s	262MB	50/s	350MB	962ms	1196ms	1366ms	1665ms	2878ms	6868ms
0.5 vCPU		49.063s	258MB	10/s	337MB	4299ms	6701ms	9344ms	16138ms	21606ms	21606ms
0.5 vCPU		50.463s	258MB	15/s	329MB	3302ms	3865ms	4414ms	7873ms	14262ms	14262ms
0.25 vCPU		108.793s	259MB	2/s	335MB	30324ms	43535ms	48836ms	56330ms	56330ms	56330ms
0.25 vCPU		107.764s	259MB	5/s	317MB	10337ms	11799ms	15141ms	20259ms	20991ms	20991ms

Virtual Threads Java 24

CPU	VT	Startup	Initial RAM	Request/s	RAM	50%	75%	90%	99%	99.9%	99.99%
4 vCPU		6.142s	275MB	326/s	523MB	133ms	204ms	288ms	459ms	603ms	734ms
4 vCPU		6.509s	275MB	802/s	483MB	56ms	87ms	120ms	190ms	262ms	341ms
2 vCPU		8.335s	251MB	87/s	394MB	497ms	797ms	1136ms	1994ms	2742ms	3369ms
2 vCPU		8.528s	255MB	702/s	349MB	53ms	80ms	100ms	420ms	2055ms	2976ms
1 vCPU		23.021s	253MB	24/s	343MB	1967ms	2834ms	3768ms	5804ms	7654ms	8270ms
1 vCPU		24.312s	253MB	188/s	342MB	222ms	304ms	493ms	875ms	1186ms	1221ms
0.5 vCPU		49.831s	248MB	10/s	316MB	4467ms	6894ms	9695ms	15111ms	19059ms	19059ms
0.5 vCPU		50.501s	251MB	20/s	317MB	2272ms	2769ms	3471ms	5870ms	9721ms	12095ms
0.25 vCPU		112.161s	250MB	2/s	312MB	48899ms	53993ms	56430ms	58705ms	58705ms	58705ms
0.25 vCPU		108.235s	246MB	6/s	295MB	9114ms	10166ms	12534ms	17828ms	21275ms	21275ms

Virtual Threads Java 25

CPU	VT	Startup	Initial RAM	Request/s	RAM	50%	75%	90%	99%	99.9%	99.99%
4 vCPU		6.002s	288MB	390/s	390MB	114ms	150ms	181ms	194ms	621ms	2346ms
4 vCPU		6.236s	277MB	627/s	669MB	64ms	116ms	172ms	283ms	416ms	587ms
2 vCPU		8.359s	262MB	164/s	347MB	296ms	332ms	400ms	572ms	868ms	2204ms
2 vCPU		8.198s	258MB	727/s	362MB	53ms	63ms	95ms	190ms	2753ms	3985ms
1 vCPU		23.159s	262MB	50/s	350MB	962ms	1196ms	1366ms	1665ms	2878ms	6868ms
1 vCPU		22.67s	255MB	195/s	342MB	219ms	301ms	411ms	896ms	1203ms	1279ms
0.5 vCPU		50.463s	258MB	15/s	329MB	3302ms	3865ms	4414ms	7873ms	14262ms	14262ms
0.5 vCPU		49.13s	251MB	21/s	317MB	1936ms	2533ms	3282ms	4849ms	20245ms	21156ms
0.25 vCPU		107.764s	259MB	5/s	317MB	10337ms	11799ms	15141ms	20259ms	20991ms	20991ms
0.25 vCPU		108.601s	253MB	6/s	303MB	8801ms	9901ms	11161ms	18049ms	28540ms	28540ms

Summary

Summary

- No Optimisations with JDK 17 & JDK 21, ...
- JVM Tuning
- Lazy Spring Beans
- No Spring Boot Actuators
- Fix Spring Boot Config Location
- Disabling JMX
- Dependency Cleanup
- Ahead-of-Time Processing (AOT)
- JLink
- Other JVMs (Eclipse OpenJ9, GraalVM, OpenJDK with CRaC)
- GraalVM Native Image

Conclusions

Conclusions (1)

CPUs

- Your application may not need a full CPU at runtime.
- It will need several CPUs to start as fast as possible (at least 2, 4 is better).
- If you don't mind a slower startup, you can throttle the CPUs below 4.

See: <https://spring.io/blog/2018/11/08/spring-boot-in-a-container>

Conclusions (2)

Request/s

- Every application is different and has different requirements.
- Proper load testing can help find the optimal configuration for your application.

Conclusions (3)

Other Runtimes

- CRIU Support for OpenJDK and OpenJ9 is promising.
 - Supported by Spring since Spring Boot 3.2 / Spring Framework 6.1
- GraalVM Native Image is a great option for Java applications
 - But build times are long
 - The result is different from what you run in your IDE
- Eclipse OpenJ9 is an excellent option for running applications with less memory
 - But startup times are longer than with HotSpot.
- Depending on the distribution, you may get other exciting features.
 - Oracle GraalVM Enterprise Edition, Azul Platform Prime, IBM Semeru Runtime, ...

Conclusions (4)

Other Ideas

- CRaC (Coordinated Restore at Checkpoint)*
- Tree shaking / Obfuscator such as ProGuard*
- More JVM tuning (GC, Memory, etc.)
- Project Leyden

A Few Simple Optimisations Applied

```
1  def fib(n):  
2      if n < 2:  
3          return n  
4      return fib(n-1) + fib(n-2)  
5  
6  def fib2(n):  
7      a, b = 0, 1  
8      for i in range(n):  
9          a, b = b, a + b  
10     return a  
11  
12  def fib3(n):  
13     if n < 2:  
14         return n  
15     return fib3(n-2) + fib3(n-1)  
16  
17  def fib4(n):  
18     if n < 2:  
19         return n  
20     return fib4(n-1) + fib4(n-2)  
21  
22  def fib5(n):  
23     if n < 2:  
24         return n  
25     return fib5(n-1) + fib5(n-2)  
26  
27  def fib6(n):  
28     if n < 2:  
29         return n  
30     return fib6(n-1) + fib6(n-2)  
31  
32  def fib7(n):  
33     if n < 2:  
34         return n  
35     return fib7(n-1) + fib7(n-2)  
36  
37  def fib8(n):  
38     if n < 2:  
39         return n  
40     return fib8(n-1) + fib8(n-2)  
41  
42  def fib9(n):  
43     if n < 2:  
44         return n  
45     return fib9(n-1) + fib9(n-2)  
46  
47  def fib10(n):  
48     if n < 2:  
49         return n  
50     return fib10(n-1) + fib10(n-2)  
51  
52  def fib11(n):  
53     if n < 2:  
54         return n  
55     return fib11(n-1) + fib11(n-2)  
56  
57  def fib12(n):  
58     if n < 2:  
59         return n  
60     return fib12(n-1) + fib12(n-2)  
61  
62  def fib13(n):  
63     if n < 2:  
64         return n  
65     return fib13(n-1) + fib13(n-2)  
66  
67  def fib14(n):  
68     if n < 2:  
69         return n  
70     return fib14(n-1) + fib14(n-2)  
71  
72  def fib15(n):  
73     if n < 2:  
74         return n  
75     return fib15(n-1) + fib15(n-2)  
76  
77  def fib16(n):  
78     if n < 2:  
79         return n  
80     return fib16(n-1) + fib16(n-2)  
81  
82  def fib17(n):  
83     if n < 2:  
84         return n  
85     return fib17(n-1) + fib17(n-2)  
86  
87  def fib18(n):  
88     if n < 2:  
89         return n  
90     return fib18(n-1) + fib18(n-2)  
91  
92  def fib19(n):  
93     if n < 2:  
94         return n  
95     return fib19(n-1) + fib19(n-2)  
96  
97  def fib20(n):  
98     if n < 2:  
99         return n  
100    return fib20(n-1) + fib20(n-2)
```

A Few Simple Optimisations Applied

- Dependency Cleanup
 - DB Drivers, Spring Boot Actuator, Jackson, Tomcat Websocket, ...
- Bellsoft Builder (musl) / Base Builder Tiny
- JLink
- JVM Parameters (java-memory-calculator)
- Application Class Data Sharing (AppCDS)
- Spring AOT
- Lazy Spring Beans
- Fixing Spring Boot Config Location
- Virtual Threads

Bellsoft Builder (musl)

```

sdk use java 21.0.9-librca

mvn spring-boot:build-image

docker run -p 8080:8080 \
  -e spring.threads.virtual.enabled=true \
  -e spring.main.lazy-initialization=true \
  -e spring.data.jpa.repositories.bootstrap-mode=lazy \
  -e spring.config.location=classpath:application.properties \
  -t spring-petclinic-optimized-builder-bellsoft-musl:4.0.0-SNAPSHOT

```

	VT	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17		76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
Java 17		84s	149MB	2.828s	225MB	409/s	363MB	105ms	183ms	200ms	300ms	401ms
Java 21		88s	218MB	2.522s	230MB	523/s	371MB	98ms	108ms	182ms	208ms	298ms
Java 25		76s	192MB	2.829s	235MB	315/s	658MB	183ms	199ms	214ms	306ms	399ms

Java 21 & Base Builder Tiny

	VT	AC	COH	Build	Image	Startup	I RAM	R/s	RAM	50%	75%	90%	99%	99.9%
 Java 17				76s	277MB	4.503s	263MB	338/s	374MB	116ms	197ms	251ms	386ms	489ms
Java 17				97s	315MB	2.798s	216MB	447/s	357MB	102ms	155ms	196ms	295ms	393ms
Java 21				96s	322MB	2.708s	221MB	490/s	362MB	100ms	111ms	187ms	280ms	309ms
Java 25				95s	323MB	2.69s	216MB	346/s	661MB	122ms	192ms	203ms	294ms	380ms
Java 25				96s	290MB	3.052s	219MB	318/s	641MB	183ms	198ms	210ms	298ms	385ms
Java 25				(96s)	(290MB)	2.905s	216MB	318/s	646MB	183ms	198ms	210ms	298ms	386ms
Java 25				91s	482MB	2.069s	208MB	279/s	354MB	193ms	209ms	289ms	381ms	405ms
Java 25				(91s)	(482MB)	1.964s	207MB	278/s	355MB	193ms	209ms	290ms	384ms	408ms

Did I miss something? 🧐

Let me/us know! 🙋

... or get in touch later! 🙈

Spring Boot in the Cloud

Advanced Optimization Deep Dive

Patrick Baumgartner

42talents GmbH, Zürich, Switzerland

@patbaumgartner

patrick.baumgartner@42talents.com

bit.ly/48i5U9x

